

COMP0264 Generative AI

Exam Revision Notes: Diffusion Models

Auto-generated from Lecture Slides & Recordings

UCL 2025–26

Contents

1 Overview	2
2 Notation and Setup	2
3 Forward Encoder (Adding Noise)	3
3.1 Single-Step Noise Addition	3
3.2 Multi-Step Forward Process	3
4 Diffusion Kernel	4
5 Reversing the Forward Process	5
5.1 Conditional Reverse Distribution	5
5.2 Result: Product of Two Gaussians	5
6 Reverse Decoder (Learning to Denoise)	6
6.1 Why We Need a Neural Network	6
6.2 Neural Network Parameterisation	6
6.3 Overall Reverse Process	7
6.4 Generation Procedure	7
7 Training via ELBO	7
7.1 Likelihood is Intractable	7
7.2 Evidence Lower Bound (ELBO)	7
7.3 ELBO Decomposition into Reconstruction and Consistency Terms	8
7.4 Closed-Form KL Divergence	8
8 Predicting Noise Instead of the Mean	9
9 Extensions and Variations	9
10 Summary of Key Formulas	10
11 Potential Exam Questions	10
12 References	11

1 Overview

This topic was covered in **one lecture** (2026-03-23) with the corresponding slide deck *GenAI2526_DiffusionModels*. The lecturer described this as “one of the most complex topics in modern AI/machine learning” and emphasized the need for revision beyond the lecture itself.

Diffusion models (also called **denoising diffusion probabilistic models**, DDPMs) are a class of generative models that learn to generate data (typically images) by reversing a gradual noise-corruption process. The overall framework consists of two processes:

1. **Forward process** (encoder): progressively adds Gaussian noise to a data sample until it becomes indistinguishable from pure noise.
2. **Reverse process** (decoder): a learned neural network that iteratively denoises, transforming noise samples back into data.

LECTURE INSIGHT:

General Motivation The lecturer stressed that diffusion models are “semi-magical”—they capture aspects of reality using probability distributions applied repeatedly. The same principles power video generation systems like Video 3 (based on diffusion models) that can capture physics of the real world. “You are able to capture reality using a probability distribution applied several times. This is not obvious—it means there is some order.”

LECTURE INSIGHT:

Historical Context Two key papers:

1. **Sohl-Dickstein et al. (ICML 2015)**: “Deep Unsupervised Learning Using Nonequilibrium Thermodynamics”—the theoretical foundation, rooted in statistical mechanics. The lecturer described it as “pretty dark” and primarily of interest to those with a statistical mechanics background.
2. **Ho et al. (NeurIPS 2020)**: “Denoising Diffusion Probabilistic Models”—the paper that made diffusion models practical with excellent results. Described as “more accessible” and recommended reading.

Additionally, around the same time as the 2020 paper, researchers at Stanford (Yang Song et al.) presented an equivalent formulation using **score matching functions**—an alternative but mathematically equivalent approach.

2 Notation and Setup

Notation

- $\mathbf{x}$: observed data vector (e.g. an image from the training set).
- $\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_T$: sequence of increasingly noisy latent variables.
- $q(\mathbf{z}_t | \mathbf{z}_{t-1})$: forward (noise-adding) conditional distribution.
- $p(\mathbf{z}_{t-1} | \mathbf{z}_t, \mathbf{w})$: learned reverse (denoising) conditional distribution with neural network weights $\mathbf{w}$.
- $q(\mathbf{z}_{t-1} | \mathbf{z}_t, \mathbf{x})$: conditional reverse distribution given the original data—plays a key role in training.
- $\beta_t \in (0, 1)$: noise variance schedule, with $\beta_1 < \beta_2 < \dots < \beta_T$.
- $\alpha_t = \prod_{\tau=1}^t (1 - \beta_\tau)$: cumulative product of noise retention factors.

LECTURE INSIGHT:

Notation Conventions The lecturer uses the notation from Bishop & Bishop’s textbook, where $\mathbf{z}_t$ denotes the latent (noisy) variable at step t . Some other references use $\mathbf{x}_t$ instead of $\mathbf{z}_t$ for the

intermediate noisy images. The key distinction is that $\mathbf{z}_t$ are *latent variables*, while $\mathbf{x}$ lives in the data space.

3 Forward Encoder (Adding Noise)

3.1 Single-Step Noise Addition

Forward Step Definition

Starting from a training image $\mathbf{x}$, we add independent Gaussian noise to each pixel to obtain the first noisy image:

$$\mathbf{z}_1 = \sqrt{1 - \beta_1} \mathbf{x} + \sqrt{\beta_1} \boldsymbol{\epsilon}_1, \quad \boldsymbol{\epsilon}_1 \sim \mathcal{N}(\boldsymbol{\epsilon}_1 \mid \mathbf{0}, \mathbf{I})$$

where β_1 is the variance of the noise distribution. This can be rewritten as:

$$q(\mathbf{z}_1 \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}_1 \mid \sqrt{1 - \beta_1} \mathbf{x}, \beta_1 \mathbf{I})$$

LECTURE INSIGHT:

Why These Coefficients? The choice of $\sqrt{1 - \beta_1}$ and $\sqrt{\beta_1}$ as coefficients ensures two properties as the process repeats:

- The **mean** of $\mathbf{z}_t$ drifts closer to $\mathbf{0}$ with each step.
- The **variance** of $\mathbf{z}_t$ drifts closer to the identity matrix $\mathbf{I}$ with each step.

The lecturer emphasised: “Because of convergence—we want the mean to converge to zero and the variance to converge to one.”

3.2 Multi-Step Forward Process

General Forward Step

We repeat the noise addition process for T steps. Each successive noisy output is given by:

$$\mathbf{z}_t = \sqrt{1 - \beta_t} \mathbf{z}_{t-1} + \sqrt{\beta_t} \boldsymbol{\epsilon}_t, \quad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\boldsymbol{\epsilon}_t \mid \mathbf{0}, \mathbf{I})$$

Equivalently:

$$q(\mathbf{z}_t \mid \mathbf{z}_{t-1}) = \mathcal{N}(\mathbf{z}_t \mid \sqrt{1 - \beta_t} \mathbf{z}_{t-1}, \beta_t \mathbf{I})$$

The noise schedule satisfies $\beta_1 < \beta_2 < \dots < \beta_T$. This defines the **forward process**.

LECTURE INSIGHT:

Markov Property The forward process is **Markovian**: $q(\mathbf{z}_t \mid \mathbf{z}_{t-1})$ depends only on the previous state $\mathbf{z}_{t-1}$, not on the full history. The lecturer: “This is a Markov process—each step depends only on the previous state.”

STUDENT Q&A:

Why does β_t increase? **Q:** Why are the β_t chosen to be increasing?

A: Because $1 - \beta_t$ gets smaller as β_t grows, which means the signal component $\sqrt{1 - \beta_t} \mathbf{z}_{t-1}$ shrinks progressively. This guarantees convergence to pure Gaussian noise. The lecturer: “ $1 - \beta_t$ gets more [small] because $\beta_3, \beta_4, \beta_5$ gets bigger.”

STUDENT Q&A:

Choice of β_t in Practice **Q**: How are the β_t values selected in practice?

A: In practice, people select values of β_t by trial and error, considering the trade-off between convergence speed and computational cost. The process is “very, very expensive”—all β_t values are hyperparameters that cost significant compute to tune. The lecturer referred to them as “the secret sauce” and noted that tuning “costs a lot of cloud money.”

4 Diffusion Kernel

Joint Forward Distribution

The joint distribution of the latent variables conditioned on the observed data $\mathbf{x}$ is:

$$q(\mathbf{z}_1, \dots, \mathbf{z}_t \mid \mathbf{x}) = q(\mathbf{z}_1 \mid \mathbf{x}) \prod_{\tau=2}^t q(\mathbf{z}_\tau \mid \mathbf{z}_{\tau-1})$$

This follows directly from the Markov property of the forward process.

Diffusion Kernel — Marginalising Intermediate Variables

If we marginalise over the intermediate variables $\mathbf{z}_1, \dots, \mathbf{z}_{t-1}$, we obtain the **diffusion kernel**:

$$q(\mathbf{z}_t \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}_t \mid \sqrt{\alpha_t} \mathbf{x}, (1 - \alpha_t) \mathbf{I})$$

where

$$\alpha_t = \prod_{\tau=1}^t (1 - \beta_\tau)$$

This can equivalently be written as:

$$\mathbf{z}_t = \sqrt{\alpha_t} \mathbf{x} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_t, \quad \boldsymbol{\epsilon}_t \sim \mathcal{N}(\boldsymbol{\epsilon}_t \mid \mathbf{0}, \mathbf{I})$$

LECTURE INSIGHT:

Key Insight of the Diffusion Kernel The diffusion kernel allows us to jump directly from the original image $\mathbf{x}$ to any noisy version $\mathbf{z}_t$ in *one step*, without computing all intermediate $\mathbf{z}_1, \dots, \mathbf{z}_{t-1}$. This is crucial for efficient training. The rewriting as $\mathbf{z}_t = \sqrt{\alpha_t} \mathbf{x} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_t$ is “just rewriting as a function of $\boldsymbol{\epsilon}_t$ ”—not adding more noise.

Convergence to Pure Noise

After sufficiently many steps, the output becomes indistinguishable from Gaussian noise. In the limit $T \rightarrow \infty$:

$$q(\mathbf{z}_T \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}_T \mid \mathbf{0}, \mathbf{I})$$

All information about the original image $\mathbf{x}$ is lost. Since the right-hand side is independent of $\mathbf{x}$, the marginal distribution is:

$$q(\mathbf{z}_T) = \mathcal{N}(\mathbf{z}_T \mid \mathbf{0}, \mathbf{I})$$

5 Reversing the Forward Process

5.1 Conditional Reverse Distribution

Reverse via Bayes' Theorem

We consider the conditional reverse distribution $q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x})$, conditioned on the data vector $\mathbf{x}$. By Bayes' theorem:

$$q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x}) = \frac{q(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}) q(\mathbf{z}_{t-1} \mid \mathbf{x})}{q(\mathbf{z}_t \mid \mathbf{x})}$$

Since $q(\mathbf{z}_t \mid \mathbf{x})$ depends only on $\mathbf{x}$ and $\mathbf{z}_t$ (both given), it acts as a normalising constant:

$$q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x}) \propto q(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}) q(\mathbf{z}_{t-1} \mid \mathbf{x})$$

Using the Markov property $q(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}) = q(\mathbf{z}_t \mid \mathbf{z}_{t-1})$:

$$q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x}) \propto q(\mathbf{z}_t \mid \mathbf{z}_{t-1}) q(\mathbf{z}_{t-1} \mid \mathbf{x})$$

LECTURE INSIGHT:

Why Condition on $\mathbf{x}$? The lecturer emphasised this is a crucial “trick”: reversing the forward process directly via $q(\mathbf{z}_{t-1} \mid \mathbf{z}_t)$ is **intractable** because it would require integrating over all possible starting images $\mathbf{x}$. Conditioning on $\mathbf{x}$ simplifies the problem because “you have the image—the image is given.”

5.2 Result: Product of Two Gaussians

Closed-Form Reverse Conditional

The right-hand side of $q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x}) \propto q(\mathbf{z}_t \mid \mathbf{z}_{t-1}) q(\mathbf{z}_{t-1} \mid \mathbf{x})$ is a product of two Gaussian distributions. Multiplying two Gaussians yields a new Gaussian. The result is:

$$q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x}) = \mathcal{N}(\mathbf{z}_{t-1} \mid \mathbf{m}_t(\mathbf{x}, \mathbf{z}_t), \sigma_t^2 \mathbf{I})$$

where

$$\mathbf{m}_t(\mathbf{x}, \mathbf{z}_t) = \frac{(1 - \alpha_{t-1})\sqrt{1 - \beta_t} \mathbf{z}_t + \sqrt{\alpha_{t-1}} \beta_t \mathbf{x}}{1 - \alpha_t}$$

and

$$\sigma_t^2 = \frac{\beta_t(1 - \alpha_{t-1})}{1 - \alpha_t}$$

with $\alpha_t = \prod_{\tau=1}^t (1 - \beta_\tau)$.

LECTURE INSIGHT:

Derivation Hint: Completing the Square The full derivation requires multiplying two Gaussian PDFs and collecting terms. The lecturer mentioned using the **completing the square** method: “The way I did it was with completing the square method.” He noted the derivation takes “some pages of calculation” and encouraged students to try it as an exercise. He described it as “not impossible, but you need to do the multiplication between the Gaussians.”

STUDENT Q&A:

Are the α_t consistent with earlier definitions? **Q:** Are the alphas here the same as the ones defined earlier?

A: Yes. $\alpha_t = \prod_{\tau=1}^t (1 - \beta_\tau)$. Note that $\alpha_{t-1} = \prod_{\tau=1}^{t-1} (1 - \beta_\tau)$ —the product is taken up to $t - 1$.

6 Reverse Decoder (Learning to Denoise)

6.1 Why We Need a Neural Network

Intractability of Direct Reversal

Directly computing $q(\mathbf{z}_{t-1} | \mathbf{z}_t)$ (without conditioning on $\mathbf{x}$) is **intractable**: it would require integrating over all possible values of $\mathbf{x}$. Moreover, we cannot sample $\mathbf{x}$ since that is the vector we want to generate in the first place.

LECTURE INSIGHT:

Why Not Just Sample $\mathbf{x}$? The lecturer explained clearly: “We cannot sample $\mathbf{x}$ because $\mathbf{x}$ is what we want to generate. When using the model for generation, we start from pure noise— $\mathbf{x}$ is not available because it is the thing we want to produce.” This motivates learning a neural network approximation.

6.2 Neural Network Parameterisation

Learned Reverse Distribution

We learn a distribution $p(\mathbf{z}_{t-1} | \mathbf{z}_t, \mathbf{w})$ governed by a deep neural network with parameters $\mathbf{w}$:

$$p(\mathbf{z}_{t-1} | \mathbf{z}_t, \mathbf{w}) = \mathcal{N}(\mathbf{z}_{t-1} | \boldsymbol{\mu}(\mathbf{z}_t, \mathbf{w}, t), \beta_t \mathbf{I})$$

where $\boldsymbol{\mu}(\mathbf{z}_t, \mathbf{w}, t)$ is the output of the neural network. Key design choices:

- The network takes the step index t as an explicit input, since β_t varies across steps.
- A **single network** is used for all steps of the chain.
- The architecture $\boldsymbol{\mu}(\cdot)$ is typically a **U-Net**.

LECTURE INSIGHT:

Why Include t as Input? The lecturer gave an important “design pattern” insight: since β_t changes at every step, the network needs to know which step it is at. “Each step is characterised by a specific value of β_t , so this network will have as input also t .” This is a general ML design principle: “If you have information, why not use it?” Using t avoids needing to train a separate network for each step.

LECTURE INSIGHT:

U-Net Architecture The network $\boldsymbol{\mu}$ is typically a **U-Net**—a specific architecture for vision tasks (segmentation, generation) with an encoder-decoder structure that “goes down and then back up.” These are the gold standard for image diffusion models.

6.3 Overall Reverse Process

Markov Chain Formulation

The overall reverse denoising process is a Markov chain:

$$p(\mathbf{x}, \mathbf{z}_1, \dots, \mathbf{z}_T | \mathbf{w}) = p(\mathbf{z}_T) \left\{ \prod_{t=2}^T p(\mathbf{z}_{t-1} | \mathbf{z}_t, \mathbf{w}) \right\} p(\mathbf{x} | \mathbf{z}_1, \mathbf{w})$$

We assume $p(\mathbf{z}_T) = q(\mathbf{z}_T) = \mathcal{N}(\mathbf{z}_T | \mathbf{0}, \mathbf{I})$.

6.4 Generation Procedure

Sampling / Image Generation

Once the model is trained, generation proceeds as follows:

1. Sample $\mathbf{z}_T \sim p(\mathbf{z}_T) = \mathcal{N}(\mathbf{z}_T | \mathbf{0}, \mathbf{I})$ (pure noise).
2. For $t = T, T-1, \dots, 2$: sample $\mathbf{z}_{t-1} \sim p(\mathbf{z}_{t-1} | \mathbf{z}_t, \mathbf{w})$ (in latent space).
3. For $t = 1$: sample $\mathbf{x} \sim p(\mathbf{x} | \mathbf{z}_1, \mathbf{w})$ (jump to data space).

LECTURE INSIGHT:

Iterative Sampling The lecturer stressed: “We sample many times—each step is a sampling process. Each time we sample, the image should get better. The input at each step is the image generated at the previous step.” It is essentially an **iteration**: sample, feed the result back as input, repeat.

7 Training via ELBO

7.1 Likelihood is Intractable

Likelihood Function

The likelihood for a data point $\mathbf{x}$ is:

$$p(\mathbf{x} | \mathbf{w}) = \int \cdots \int p(\mathbf{x}, \mathbf{z}_1, \dots, \mathbf{z}_T | \mathbf{w}) d\mathbf{z}_1 \cdots d\mathbf{z}_T$$

This is **intractable**: it requires integrating over all possible trajectories through the latent space, and the integrand involves highly complex neural network functions.

7.2 Evidence Lower Bound (ELBO)

ELBO Decomposition

For any choice of distribution $q(\mathbf{z})$, the following holds:

$$\ln p(\mathbf{x} | \mathbf{w}) = -\mathcal{L}(\mathbf{w}) + \text{KL}(q(\mathbf{z}) \| p(\mathbf{z} | \mathbf{x}, \mathbf{w}))$$

where $\mathcal{L}(\mathbf{w})$ is the ELBO:

$$\mathcal{L}(\mathbf{w}) = \int q(\mathbf{z}) \ln \left\{ \frac{p(\mathbf{x}, \mathbf{z} | \mathbf{w})}{q(\mathbf{z})} \right\} d\mathbf{z}$$

and the Kullback-Leibler divergence $\text{KL}(f(\mathbf{z})\|g(\mathbf{z}))$ is defined by:

$$\text{KL}(f(\mathbf{z})\|g(\mathbf{z})) = - \int f(\mathbf{z}) \ln \left\{ \frac{g(\mathbf{z})}{f(\mathbf{z})} \right\} d\mathbf{z}$$

Since $\text{KL}(\cdot\|\cdot) \geq 0$, we have:

$$\ln p(\mathbf{x} \mid \mathbf{w}) \geq \mathcal{L}(\mathbf{w})$$

Maximising the ELBO also maximises the log-likelihood.

LECTURE INSIGHT:

Intuition for ELBO The lecturer's simple explanation: "Informally, log-likelihood = ELBO + KL. Since $\text{KL} > 0$, log-likelihood $\geq$ ELBO. So if you maximise the ELBO, you maximise the log-likelihood. Why? Because if you maximise something that is always smaller, the bigger thing gets maximised too." This is the same approach used for **Variational Autoencoders**.

7.3 ELBO Decomposition into Reconstruction and Consistency Terms

Two-Term ELBO

The ELBO can be decomposed as:

$$\begin{aligned} \mathcal{L}(\mathbf{w}) = & \underbrace{\int q(\mathbf{z}_1 \mid \mathbf{x}) \ln p(\mathbf{x} \mid \mathbf{z}_1, \mathbf{w}) d\mathbf{z}_1}_{\text{Reconstruction term}} \\ & - \underbrace{\sum_{t=2}^T \int \text{KL}(q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x}) \parallel p(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{w})) q(\mathbf{z}_t \mid \mathbf{x}) d\mathbf{z}_t}_{\text{Consistency term}} \end{aligned}$$

Training proceeds similarly to Variational Autoencoders.

LECTURE INSIGHT:

Understanding the Two Terms

- **Reconstruction term:** How well can we reconstruct $\mathbf{x}$ from $\mathbf{z}_1$? Estimated via Monte Carlo sampling: draw samples $\mathbf{z}_1^{(l)} \sim q(\mathbf{z}_1 \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}_1 \mid \sqrt{1 - \beta_1} \mathbf{x}, \beta_1 \mathbf{I})$.
- **Consistency term:** Penalises divergence between the learned reverse $p(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{w})$ and the true reverse $q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x})$. The lecturer: "It's called consistency because we don't want to be far away from the forward process. The minus sign means we want to minimise this KL—if they are very similar, that is considered good."

7.4 Closed-Form KL Divergence

KL Between Reverse Distributions

Since both $q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x})$ and $p(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{w})$ are Gaussian, their KL divergence has a closed form:

$$\text{KL}(q(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{x}) \parallel p(\mathbf{z}_{t-1} \mid \mathbf{z}_t, \mathbf{w})) = \frac{1}{2\beta_t} \|\mathbf{m}_t(\mathbf{x}, \mathbf{z}_t) - \boldsymbol{\mu}(\mathbf{z}_t, \mathbf{w}, t)\|^2 + \text{const}$$

where $\mathbf{m}_t(\mathbf{x}, \mathbf{z}_t)$ is the true reverse mean and $\boldsymbol{\mu}(\mathbf{z}_t, \mathbf{w}, t)$ is the network prediction.

LECTURE INSIGHT:

What The Network Learns Training reduces to making the network’s predicted mean $\boldsymbol{\mu}(\mathbf{z}_t, \mathbf{w}, t)$ match the true conditional mean $\mathbf{m}_t(\mathbf{x}, \mathbf{z}_t)$. The loss is essentially a **mean squared error** between the two means, weighted by $1/(2\beta_t)$.

LECTURE INSIGHT:

Efficient Computation via Diffusion Kernel Each consistency term has an integral over $\mathbf{z}_t$ weighted by $q(\mathbf{z}_t | \mathbf{x})$. This can be computed efficiently using the diffusion kernel $q(\mathbf{z}_t | \mathbf{x}) = \mathcal{N}(\mathbf{z}_t | \sqrt{\alpha_t} \mathbf{x}, (1 - \alpha_t) \mathbf{I})$ —one can simply sample $\mathbf{z}_t$ directly from this distribution.

8 Predicting Noise Instead of the Mean

LECTURE INSIGHT:

Key Practical Trick (Ho et al. 2020) The lecturer mentioned that the original DDPM paper (Ho et al., 2020) and many practical implementations do *not* directly learn the mean $\boldsymbol{\mu}$. Instead, they learn to predict the **noise** $\boldsymbol{\epsilon}$ that was added:

- Rather than predicting $\boldsymbol{\mu}(\mathbf{z}_t, \mathbf{w}, t)$, the network predicts $\hat{\boldsymbol{\epsilon}}(\mathbf{z}_t, \mathbf{w}, t)$.
- The mean can be recovered from the predicted noise using the known relationship between $\mathbf{z}_t$, $\mathbf{x}$, and $\boldsymbol{\epsilon}$.

This is “cheaper because probably less steps are needed” and is a key contribution of the 2020 paper.

9 Extensions and Variations

Beyond Basic Diffusion

The area of diffusion models is vast. The lecture covered the key principles as a starting point for:

1. **Score matching:** An alternative formulation using the score function $\nabla_{\mathbf{z}} \log q(\mathbf{z})$. Developed concurrently by Yang Song et al. at Stanford (NeurIPS 2019).
2. **Guided diffusion:** Conditioning the generation on additional input such as a text prompt or class label. This is the basis of systems like DALL-E and Stable Diffusion.
3. **Discrete diffusion:** Generating discrete structures such as molecules or graphs.

LECTURE INSIGHT:

Guided Diffusion The lecturer explained: “You condition on a class or a conditional variable. You need a lot of examples because you need to generate images for variations of the prompt—that is really expensive.” All major text-to-image systems (DALL-E, Stable Diffusion) are based on guided diffusion.

LECTURE INSIGHT:

Stable Diffusion and Latent Diffusion **Stable Diffusion** performs the diffusion process in a lower-dimensional latent space rather than pixel space, making it computationally cheaper. “These are variations of this concept—just check Stable Diffusion and you get the key changes compared to this idea.”

10 Summary of Key Formulas

Quantity	Formula
Forward step	$q(\mathbf{z}_t \mathbf{z}_{t-1}) = \mathcal{N}(\mathbf{z}_t \sqrt{1 - \beta_t} \mathbf{z}_{t-1}, \beta_t \mathbf{I})$
Diffusion kernel	$q(\mathbf{z}_t \mathbf{x}) = \mathcal{N}(\mathbf{z}_t \sqrt{\alpha_t} \mathbf{x}, (1 - \alpha_t) \mathbf{I}), \quad \alpha_t = \prod_{\tau=1}^t (1 - \beta_\tau)$
Convergence	$q(\mathbf{z}_T) = \mathcal{N}(\mathbf{z}_T \mathbf{0}, \mathbf{I})$ as $T \rightarrow \infty$
True reverse	$q(\mathbf{z}_{t-1} \mathbf{z}_t, \mathbf{x}) = \mathcal{N}(\mathbf{z}_{t-1} \mathbf{m}_t(\mathbf{x}, \mathbf{z}_t), \sigma_t^2 \mathbf{I})$
Reverse mean	$\mathbf{m}_t = \frac{(1 - \alpha_{t-1}) \sqrt{1 - \beta_t} \mathbf{z}_t + \sqrt{\alpha_{t-1}} \beta_t \mathbf{x}}{1 - \alpha_t}$
Reverse variance	$\sigma_t^2 = \frac{\beta_t (1 - \alpha_{t-1})}{1 - \alpha_t}$
Learned reverse	$p(\mathbf{z}_{t-1} \mathbf{z}_t, \mathbf{w}) = \mathcal{N}(\mathbf{z}_{t-1} \boldsymbol{\mu}(\mathbf{z}_t, \mathbf{w}, t), \beta_t \mathbf{I})$
ELBO bound	$\ln p(\mathbf{x} \mathbf{w}) \geq \mathcal{L}(\mathbf{w})$
KL loss	$\frac{1}{2\beta_t} \ \mathbf{m}_t(\mathbf{x}, \mathbf{z}_t) - \boldsymbol{\mu}(\mathbf{z}_t, \mathbf{w}, t)\ ^2 + \text{const}$

11 Potential Exam Questions

EXAM ALERT:

Difficulty Warning The lecturer explicitly warned: “This will not be the easiest lecture of the year. Please be prepared.” and “This lecture requires revision—the concepts are not straightforward at all.” He mentioned that colleagues and other professors agree this is “one of the most complex topics in modern AI/ML.”

EXAM ALERT:

Derive the Reverse Conditional The lecturer repeatedly encouraged students to derive $q(\mathbf{z}_{t-1} | \mathbf{z}_t, \mathbf{x})$ by multiplying two Gaussians (completing the square). He said: “I would like to invite you to do it” and “try to derive it—it’s a good exercise.” This could appear as a derivation question on the exam.

EXAM ALERT:

ELBO Connection to VAEs The lecturer drew explicit parallels to the VAE ELBO: “Training is done in a similar way as for Variational Autoencoders.” Expect a question comparing the two ELBO formulations or asking why ELBO maximisation works.

EXAM ALERT:

Explain the Forward Process and Convergence Understanding why the forward process converges to $\mathcal{N}(\mathbf{0}, \mathbf{I})$, and the role of $\alpha_t \rightarrow 0$, is likely examinable. Students should be able to explain the choice of coefficients $\sqrt{1 - \beta_t}$ and $\sqrt{\beta_t}$.

EXAM ALERT:

Why a Single Network with t as Input? The design choice of using one network for all steps, with t as an explicit input, was emphasised as a key design pattern. This is a conceptual question likely to appear.

EXAM ALERT:

Mock Exam The lecturer mentioned he will provide a mock exam paper and hold a revision session (possibly around 20 April), approximately one week before the exam on 28 April. Check Moodle for announcements.

12 References

Slides References

- Chapter 20 of Bishop & Bishop, *Introduction to Deep Learning*, Springer 2024. (Primary reference; full derivations available [here](#).)
- Figures taken from Bishop & Bishop (made available by authors).

Lecture-Mentioned Papers and Resources

- Sohl-Dickstein et al., “Deep Unsupervised Learning Using Nonequilibrium Thermodynamics,” ICML 2015.
- Ho et al., “Denoising Diffusion Probabilistic Models,” NeurIPS 2020.
- Yang Song et al., “Generative Modeling by Estimating Gradients of the Data Distribution,” NeurIPS 2019. (Score matching formulation.)
- GLIDE: text-guided diffusion (OpenAI, 2022).
- Stable Diffusion / Latent Diffusion Models.